Studio Nx

백엔드 157 — decode_token_with_iat()

2026-10-14

백엔드 157 — `decode_token_with_iat()` (토큰에서 user_id와 발급 시각 꺼내기)

항목	내용
파일	`studionx/backend/app/core/security.py`
작업	P1-28 비밀번호 찾기 (FR-118): 여덟 번째 함수 (내부 도우미 `_decode_claims()` 포함)
작성	Claude (AI), 2026-10-15, 사용자 검토
이전	156 `issued_before_password_change()`
시험	`tests/test_security.py`의 14개 (이 파일 전체 45개)
다음	`get_current_user`(액세스 토큰)와 `/auth/refresh`(리프레시 토큰)에 연결

1. 하는 일

토큰을 검사하고 **(user_id, 발급 시각 `iat`)** 를 돌려줍니다. 156번의 `issued_before_password_change()`가 `iat`를 필요로 하는데, 기존 `decode_access_token()`과 `decode_refresh_token()`은 `user_id`만 돌려주기 때문에 필요한 함수입니다.

입력	결과
정상 액세스 토큰, `"access"`	`(7, 1760443200)` 같은 (user_id, iat)
정상 리프레시 토큰, `"refresh"`	같은 모양
`iat`가 없는 토큰	`(7, None)` (형식 판단은 쓰는 쪽 몫)
서명, 만료, 형식, 종류가 틀림	기존과 같은 401 "Please sign in again."
`token_type`이 `"access"`, `"refresh"`가 아님	`ValueError` (부르는 쪽의 실수)

2. 코드

```
def _decode_token(token: str, expected_type: str) -> int:
    return _decode_claims(token, expected_type)[0]          # 기존 동작 그대로

def _decode_claims(token: str, expected_type: str) -> tuple[int, object]:
    ...   # 기존 _decode_token의 검사와 같고, (int(payload["sub"]), payload.get("iat"))를 돌려줌

def decode_token_with_iat(token: str, token_type: str) -> tuple[int, object]:
    if token_type not in ("access", "refresh"):
        raise ValueError("token_type must be 'access' or 'refresh'")
    return _decode_claims(token, token_type)
```

3. 왜 이렇게

선택	이유
기존 함수의 반환값을 바꾸지 않고 **새 함수를 더함**	`decode_access_token`을 부르는 곳과 시험이 많습니다. 반환값을 `(user_id, iat)`로 바꾸면 모두 고쳐야 하고, 인증의 핵심 경로라서 위험합니다
검사 코드를 `_decode_claims()` 한 곳으로 모음	기존 함수와 새 함수가 **같은 코드**로 검사해서 서로 어긋날 수 없습니다. 같은 검사를 복사해 두면 한쪽만 고치는 실수가 생깁니다
`_decode_token()`은 껍데기로 남김	이름과 동작이 그대로라 기존 호출부와 시험이 바뀌지 않습니다
`iat`를 적힌 그대로 돌려줌	형식 검사(숫자인지, 없는지)는 `issued_before_password_change()`가 이미 합니다. 한 곳에서만 판단합니다
잘못된 `token_type`은 `ValueError`	`"ACCESS"`처럼 잘못 쓰면 모든 토큰이 401로 거부되어 원인을 찾기 어렵습니다. 부르는 쪽의 실수는 바로 알립니다

4. 시험 (14개 추가, 파일 전체 45개 통과)

확인	결과
진짜 액세스 토큰에서 user_id와 정수 `iat`를 꺼냄	✓
리프레시 토큰도 같음	✓
`iat`가 없으면 `None`	✓
토큰에 적힌 `iat`를 그대로 돌려줌	✓
기존 `decode_access_token`, `decode_refresh_token`과 같은 user_id	✓
틀린 토큰 7종(깨진 글자, 빈 문자열, 틀린 비밀키, 만료, `sub` 없음, `sub`가 숫자 아님, 종류가 다름)이 **같은 401, 같은 문구**	✓ (7개)
액세스를 리프레시로, 리프레시를 액세스로 물으면 거부	✓
`alg: none` 위조 토큰 거부	✓
잘못된 `token_type`(`""`, `"ACCESS"`, `"id"`, `None`, `1`)은 `ValueError`	✓ (5개)
156번 함수와 이어 쓰기: 같은 초는 유효, 재설정이 나중이면 무효, 재설정한 적 없으면 유효	✓

시험을 확인하는 시험. 일부러 틀리게 바꿔 시험이 잡는지 보았습니다.

바꾼 것	결과
`iat`를 항상 `None`으로	4개 실패
토큰 종류 검사 제거	5개 실패
`token_type` 검사 제거	5개 실패
기존 함수가 다른 값을 돌려주게	4개 실패
알고리즘 고정 해제(`["HS256", "none"]`)	**실패 없음**

마지막은 시험의 빈틈이 아니라 **라이브러리의 이중 방어**입니다. PyJWT는 키가 있는 상태에서 `alg: none`을 풀려고 하면 스스로 오류를 내서, 알고리즘 목록에 `none`을 넣어도 위조 토큰이 거부됩니다. 이 코드의 `algorithms=[JWT_ALGORITHM]` 고정은 그 라이브러리 동작에 기대지 않고 의도를 코드에 명시하는 것입니다.

시험을 만들며 고친 것. 새 시험이 짧은 비밀키(12바이트)를 써서 PyJWT의 키 길이 경고가 한 번 났습니다. 기존 시험처럼 32바이트 이상의 키로 바꿔서 경고를 없앴습니다.

전체 pytest 774개 통과, `ruff` 통과.